

SEKIRO → UE 5.8.2

Motion Matching 的真实时序

严格区分 Animation Update、Pose Evaluate、Root Motion 消费与 CharacterMovement

修正要点：Update 与 Evaluate 不是同一步，也不保证同线程。本项目因使用 RootMotionFromEverything，CMC 会在移动前同步要求一次可用的 AnimGraph Update，以取得本帧动画 Root Motion；Pose Evaluate 通常随后由 SkeletalMeshComponent::RefreshBoneTransforms 发起，并可在工作线程完成。

项目基线：F:/ProjectAI/Sekiro5.8

引擎源码：F:/UnrealEngine-5.8 · UE 5.8.2

修订日期：2026-09-14 · 阅读时间约 35 分钟

目录

目录	2
摘要：不要把一帧画成一条串行直线	3
1 可观察现象：为什么角色先移动，骨骼姿势还能稍后求值	3
2 先固定组件 Tick 的外层顺序	4
3 为什么本项目的 Animation Update 会被 CMC 提前	4
4 TickCharacterPose 实际只把动画更新推进到可消费 Root Motion	5
5 Mesh::TickPose 为什么仍然不是 Pose Evaluate	6
6 UAnimInstance::UpdateAnimation 内部也有两层 Update	6
7 Root Motion 在哪里产生：资产播放器 Update，而非骨骼 Evaluate	7
8 CMC 消费完成后，Mesh Tick 才发起 Pose Evaluate	7
9 Evaluate 做什么，以及它不保证什么	8
10 项目 Motion Matching 的实际单帧主链	8
11 跨帧反馈：本帧分别使用哪些“上一轮数据”	9
12 实例 A：Idle 按 W 后的一帧	10
13 实例 B：持续按 W 撞墙	10
14 哪些情况下时序会变	10
15 修正版逐帧时序表	11
16 断点验证：怎样亲自确认这三个阶段	11
17 项目代码流程索引	12
18 结论	12

摘要：不要把一帧画成一条串行直线

玩家按住 W 后，输入、动画状态更新、Root Motion 提取、CMC 移动、骨骼姿势求值和下一帧反馈虽然属于同一个游戏帧附近的工作，却不是一个不可分割的函数。UE 将它们拆成多个调度阶段，并在需要 Root Motion 时建立一个同步消费点。

阶段	回答的问题	本项目典型执行位置	能否独立/并行
游戏线程动画前置更新	本帧 Query、Phase、Chooser 候选是什么	UAnimInstance::UpdateAnimation 内的 NativeUpdateAnimation	与图 Update 分层
AnimGraph Update	哪些节点 relevant、时间如何推进、MM 选哪段	RootMotionFromEverything 时由 CMC 请求同步完成	一般可并行；本项目该路径被强制立即更新
Root Motion 提取	所选资产从旧时间到新时间走了多少	Asset Player Tick / Proxy PostUpdate	属于 Update 推进，不要求骨骼 Evaluate
CMC 消费与物理	这段位移碰撞后能走多少	CharacterMovement PerformMovement	游戏线程
Pose Evaluate	最终每根骨骼在哪里、输出什么姿势	Mesh RefreshBoneTransforms	通常可派发工作线程

本文后续使用“Update”专指 AnimInstance / AnimGraph 的状态推进，使用“Evaluate”专指骨骼 Pose 求值。`TickCharacterPose` 是函数名，不代表它完成了 Pose Evaluate。

1 可观察现象：为什么角色先移动，骨骼姿势还能稍后求值

启动 PIE，按住 W。角色胶囊在本帧 CMC 物理中前进，而 Animation Insights 中可能看到 AnimGraph Update 与 Pose Evaluate 处于不同的事件块，Evaluate 甚至运行在工作线程。这并不矛盾：CMC 当前需要的是动画时间区间产生的 Root Motion Delta，不需要先算出所有骨骼的最终矩阵。

可以把动画资产想成同时提供两种输出：一是时间区间 $[t_0, t_1]$ 的 Root Motion 曲线增量；二是 t_1 时刻的完整骨骼 Pose。前者可在资产播放器推进时提取并立刻交给 CMC，后者可稍后并行求值供渲染、IK 与 PoseHistory 使用。



图 1 根运动和骨骼姿势是同一动画状态的两种输出，但不必在同一函数或同一线程产生。

2 先固定组件 Tick 的外层顺序

2.1 InputManager → MovementComponent

```
// Source/Sekiro/Movement/SKMovementComponent.cpp
// USKMovementComponent::BeginPlay
if (InputManager)
{
    AddTickPrerequisiteComponent(InputManager);
}
```

该项目依赖保证移动 Lua 在读取输入时拿到本帧最新意图。它不规定 AnimGraph Evaluate 的线程或完成点。

2.2 CharacterMovement → SkeletalMeshComponent

```
// Engine/Source/Runtime/Engine/Private/Character.cpp
// ACharacter::PostInitializeComponents
Mesh->PrimaryComponentTick.AddPrerequisite(
    CharacterMovement,
    CharacterMovement->PrimaryComponentTick);
```

ACharacter 明确让 Mesh 的组件 Tick 排在 CharacterMovement 之后。正常可见角色的 `RefreshBoneTransforms` 因而通常在移动完成后才发起。这个依赖恰好说明：不能把 Pose Evaluate 画进 CMC 的 PerformMovement 内。

2.3 项目 Movement Tick 的局部顺序

```
void USKMovementComponent::TickComponent(...)
{
    RefreshCachedComponents();
    ...
    if (bUseLuaMovementLogic)
    {
        UpdateMovementLogic(DeltaTime); // 提交 Intent
    }
    Super::TickComponent(...); // 进入原生 CMC
}
```

3 为什么本项目的 Animation Update 会被 CMC 提前

项目 `USKAnimInstance` 在构造与初始化时设置 `RootMotionFromEverything`。这个模式不是“当前动画一定有非零位移”，而是告诉 Mesh/Character：图中所有合适的动画源都要参与 Root Motion 提取。

```
// Source/Sekiro/Animation/SKAnimInstance.cpp
USKAnimInstance::USKAnimInstance()
{
    RootMotionMode = ERootMotionMode::RootMotionFromEverything;
}

void USKAnimInstance::NativeInitializeAnimation()
{
    Super::NativeInitializeAnimation();
    RootMotionMode = ERootMotionMode::RootMotionFromEverything;
}
```

引擎的 `USkeletalMeshComponent::IsPlayingRootMotionFromEverything` 只检查此模式，因此本项目角色在 CMC 看来处于动画 Root Motion 路径，即便某一帧最终提取出的 Delta 为 0。

```
bool USkeletalMeshComponent::IsPlayingRootMotionFromEverything() const
{
    return AnimScriptInstance
        && AnimScriptInstance->RootMotionMode
        == ERootMotionMode::RootMotionFromEverything;
}
```

因此“Update 位于 CMC 内”是本项目 RootMotionFromEverything 主路径的结论，不是所有 SkeletalMesh、所有 AnimBlueprint、所有网络角色都遵循的普遍定律。

4 TickCharacterPose 实际只把动画更新推进到可消费 Root Motion

4.1 CMC 在 PerformMovement 中请求动画更新

```
// CharacterMovementComponent.cpp
// UCharacterMovementComponent::PerformMovement
if (bHasRootMotionSources
    && CharacterOwner->IsPlayingRootMotion()
    && CharacterOwner->GetMesh())
{
    TickCharacterPose(DeltaSeconds);
}

// 随后才把已提取 Root Motion 转成速度并进入物理
if (CurrentRootMotion.HasOverrideVelocity() || HasAnimRootMotion())
{
    RootMotionParams.Set(ConvertLocalRootMotionToWorld(...));
    AnimRootMotionVelocity = CalcAnimRootMotionVelocity(...);
}
```

4.2 名字叫 TickCharacterPose，但没有调用 RefreshBoneTransforms

```
void UCharacterMovementComponent::TickCharacterPose(float DeltaTime)
{
    CharacterMesh->bIsAutonomousTickPose = true;
    if (CharacterMesh->ShouldTickPose())
    {
        CharacterMesh->TickPose(DeltaTime, true); // [!] Update 入口
        FRootMotionMovementParams RootMotion =
            CharacterMesh->ConsumeRootMotion(); // [!] 取走已提取结果
        RootMotionParams.Accumulate(RootMotion);
    }
    CharacterMesh->bIsAutonomousTickPose = false;
}
```

这段函数没有`RefreshBoneTransforms`、`ParallelAnimationEvaluation`或`Evaluate\_AnyThread`。所以它建立的是“Animation Update 完成并得到有效 Root Motion”的同步点，不是“完整 Pose Evaluate 完成”的同步点。

5 Mesh::TickPose 为什么仍然不是 Pose Evaluate

```
// SkeletalMeshComponent.cpp
void USkeletalMeshComponent::TickPose(
    float DeltaTime, bool bNeedsValidRootMotion)
{
    Super::TickPose(DeltaTime, bNeedsValidRootMotion);
    if (ShouldTickAnimation())
    {
        TickAnimation(DeltaTimeForTick, bNeedsValidRootMotion);
    }
}

void USkeletalMeshComponent::TickAnimation(...)
{
    TickAnimInstances(DeltaTime, bNeedsValidRootMotion);
}

void USkeletalMeshComponent::TickAnimInstances(...)
{
    AnimScriptInstance->UpdateAnimation(
        DeltaTime, bNeedsValidRootMotion);
}
```

引擎命名中的 Pose Tick 更接近“动画时间/图状态的 Tick”。真正计算 BoneSpaceTransforms 的入口是后面的`RefreshBoneTransforms`。因此调试时在`TickCharacterPose`命中断点，不能据此判断`Evaluate\_AnyThread`已经执行。

6 UAnimInstance::UpdateAnimation 内部也有两层 Update

6.1 游戏线程前置更新

```
void UAnimInstance::UpdateAnimation(...)
{
    PreUpdateAnimation(DeltaSeconds);
    UpdateMontage(DeltaSeconds);
    NativeUpdateAnimation(DeltaSeconds);
    BlueprintUpdateAnimation(DeltaSeconds);
    ...
}
```

项目在`USKMotionMatchingAnimInstance::NativeUpdateAnimation`中消费上一轮 PostSelection 邮箱，读取最新 Intent 与 CMC 完成样本，构建 QuerySnapshot，推进 Phase，运行 Chooser，并发布线程安全 SearchRequest。此时 Motion Matching 节点本身还没有执行`UpdateAssetPlayer`。

6.2 AnimGraph Update 可以延后，也可以被强制立即执行

```
const bool bWantsImmediateUpdate =
    NeedsImmediateUpdate(DeltaSeconds, bNeedsValidRootMotion);

if (bShouldImmediateUpdate)
{
    ParallelUpdateAnimation(); // 名字含 Parallel，但这里是当前线程直接调用
    PostUpdateAnimation();
}

bool UAnimInstance::NeedsImmediateUpdate(...) const
{
    return
        (bNeedsValidRootMotion
         && RootMotionMode == RootMotionFromEverything)
        || !CanRunParallelWork()
        || ...;
}
```

本项目在 CMC 调用 `TickPose(DeltaTime, true)`，且 RootMotionMode 为 Everything，所以 Graph Update 会在当前调用栈中立即完成。普通非 Root Motion 动画则可只标记 `NeedsUpdate`，之后随 `RefreshBoneTransforms` 的并行任务完成 Graph Update。

7 Root Motion 在哪里产生：资产播放器 Update，而非骨骼 Evaluate

`FAnimInstanceProxy::UpdateAnimation` 先遍历 AnimGraph 的 Update 链，再推进 Active Asset Players。播放器已经知道上一时间、当前时间、循环、PlayRate 和权重，因此能够直接从动画 Root Motion Track 提取区间 Delta。

```
void FAnimInstanceProxy::UpdateAnimation()
{
    UpdateAnimation_WithRoot(Context, RootNode, NAME_AnimGraph);

    // [!] 推进 Sequence / BlendStack 等资产播放器；提取时间区间 Root Motion
    Sync.TickAssetPlayerInstances(*this, CurrentDeltaSeconds);
}
```

Motion Matching 节点的 `UpdateAssetPlayer` 在 Update 链中完成搜索、选择、BlendTo 和 BlendStack 播放器更新：

```
// PoseSearch/Private/AnimNode_MotionMatching.cpp
void FAnimNode_MotionMatching::UpdateAssetPlayer(...)
{
    // 查询 PoseHistory，执行搜索，得到 SearchResult
    ...
    FAnimNode_BlendStack_Standalone::BlendTo(...);
    FAnimNode_BlendStack_Standalone::UpdateAssetPlayer(Context);
}
```

`PostUpdateAnimation` 再把 Proxy 的 `ExtractedRootMotion` 累加到 UAnimInstance 的消费缓冲。随后 `CharacterMesh->ConsumeRootMotion()` 从 AnimInstance 取走它。整个路径不要求先生成最终骨骼数组。

```
void UAnimInstance::PostUpdateAnimation()
{
    Proxy.PostUpdate(this);
    if (Proxy.GetExtractedRootMotion().bHasRootMotion)
    {
        ExtractedRootMotion.Accumulate(
            Proxy.GetExtractedRootMotion().GetRootMotionTransform());
        Proxy.GetExtractedRootMotion().Clear();
    }
}

FRootMotionMovementParams USkeletalMeshComponent::ConsumeRootMotion_Internal(...)
{
    RootMotion.Accumulate(
        AnimScriptInstance->ConsumeExtractedRootMotion(InAlpha));
    return RootMotion;
}
```

8 CMC 消费完成后，Mesh Tick 才发起 Pose Evaluate

ACharacter 已把 Mesh Tick 设置为 CharacterMovement 的后继。因此本项目常规本地角色在 CMC 完成 Root Motion 物理移动后进入 `USkinnedMeshComponent::TickComponent`。

```
// SkinnedMeshComponent.cpp
void USkinnedMeshComponent::TickComponent(...)
{
    ...
    if (ShouldTickPose())
    {
        TickPose(DeltaTime, false);
    }

    if (ShouldUpdateTransform(bLODHasChanged))
    {
        RefreshBoneTransforms(ThisTickFunction); // [!] Evaluate 入口
    }
}
```

由于 CMC 已通过 Autonomous Tick 在本帧调用过 `TickPose`，`PoseTickedThisFrame()` 通常使 Mesh Tick 不再重复 Update；但 `RefreshBoneTransforms` 仍可发起 Evaluate。Update 与 Evaluate 因而在外层组件顺序上被明确拆开。

8.1 Evaluate 通常是并行任务

```
void USkeletalMeshComponent::RefreshBoneTransforms(...)
{
    const bool bDoParallelEvaluation =
        bHasValidInstanceForParallelWork
        && bDoPAE
        && bShouldDoEvaluation
        && TickFunction->IsCompletionHandleValid();

    if (bDoParallelEvaluation)
    {
        DispatchParallelEvaluationTasks(TickFunction);
    }
    else
    {
        DoParallelEvaluationTasks_OnGameThread();
    }
}
```

工作线程路径最终执行 `ParallelAnimationEvaluation` → `PerformAnimationProcessing`，调用各 AnimNode 的 `Evaluate\_AnyThread`，计算 BoneSpaceTransforms、曲线、属性与组件空间骨骼。完成后游戏线程 `PostAnimEvaluation` 交换缓冲并 Finalize。

9 Evaluate 做什么，以及它不保证什么

Evaluate 的输出	用途	是否直接决定本帧 CMC 位移
最终 Bone Pose	渲染、蒙皮、Socket、IK 后姿势	否；CMC 已在此前消费 Root Motion
Curve / Custom Attributes	后处理、节点间数据、调试	通常否；取决于特殊图节点和消费链
PoseSearchHistoryCollector 更新	把最终 Pose 写入历史，供后续 MM 查询	供下一轮 Update 使用
Root Motion Provider 属性	某些 Warping / MultiAnim 节点可在 Evaluate 中读取或覆盖属性	不能泛化为本项目 CMC 当前帧 Root Motion 来源

UE5.8 的 `FAnimNode\_MotionMatching::Evaluate\_AnyThread` 确实包含针对 MultiAnimAsset 对齐的 RootMotionProvider 属性处理。这说明不能反向声称“Evaluate 永远与 Root Motion 无关”。准确说法是：本项目常规 Sequence Motion Matching 的 CMC 消费缓冲，在强制立即 Update 的资产播放器推进阶段形成；Pose Evaluate 不是该消费点的前置条件。

本项目把 Orientation Warping 设为 Manual，只做骨骼姿势补偿，不依赖 Evaluate 再改写 CMC 当前帧胶囊位移，正是为了维持单一 Root Motion 所有权。

10 项目 Motion Matching 的实际单帧主链

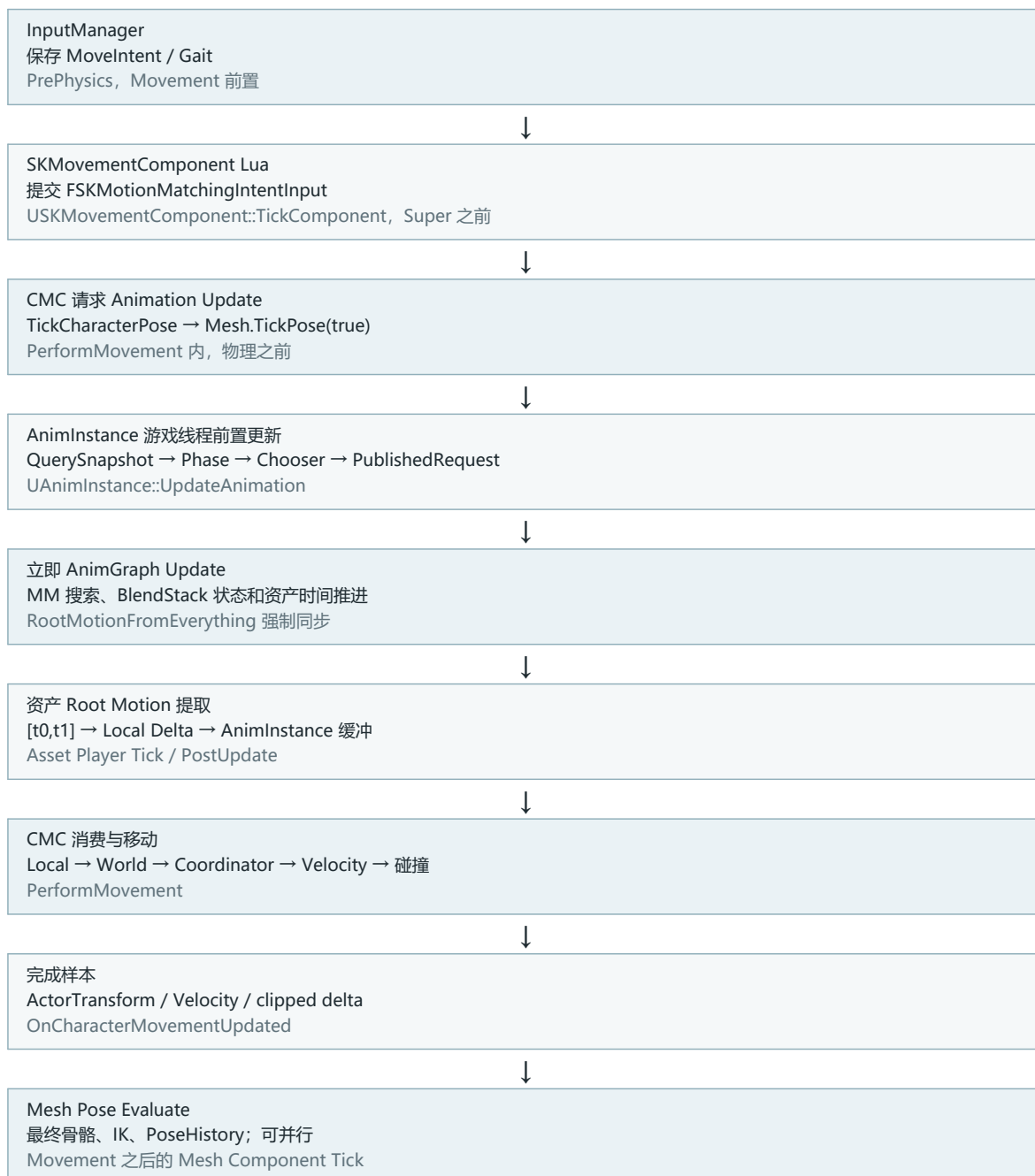


图 2 本项目主路径：Update 和 Root Motion 消费位于 CMC 同步区；Evaluate 是移动后的独立阶段。

11 跨帧反馈：本帧分别使用哪些“上一轮数据”

本帧阶段	读取的数据	数据真正来自
NativeUpdateAnimation	CompletedActualState[N-1]	上一轮 CMC 碰撞完成回调
Motion Matching Update	PoseHistory[N-1]	上一轮 Pose Evaluate / HistoryCollector
Motion Matching Update	Intent[N]	本帧 Input + Movement Lua
CMC Movement	ExtractedRootMotion[N]	本帧同步 AnimGraph Update / Asset Player Tick
Pose Evaluate	BlendStack state[N]	本帧 Motion Matching Update 已确定的播放器状态
下一帧确认	PostSelection[N] + ActualState[N]	本帧节点 Update 与 CMC 完成结果

Motion Matching 的查询闭环是跨帧的：本帧 Query 使用上一轮实际移动和 PoseHistory；本帧选择立即产生 Root Motion 并移动；本帧 Evaluate 更新可视姿势与新 PoseHistory；下一帧再确认选择和实际运动。

12 实例 A: Idle 按 W 后的一帧

顺序	代码位置	示例值 / 结果	类别
1	SKInputManager.lua	Move=(0,1), Amount=1, Gait=Run	输入
2	SKMovementComponent.lua	旋转到 WorldDir, Submit IntentRevision=42	输入
3	CMC TickCharacterPose	请求 bNeedsValidRootMotion=true	同步点
4	NativeUpdateAnimation	ActualSample=3114; Phase=StartRequested; Chooser=RunStarts	游戏线程前置 Update
5	MM UpdateAssetPlayer	选 Forward Start; BlendStack t0→t1	AnimGraph Update
6	Asset Player Tick	提取 Local RM, 例如 2.6 cm	Root Motion 提取
7	CMC PerformMovement	协调后请求 2.6 cm; 碰撞后实际 2.5 cm	移动
8	CaptureCompletedMovement	ActualSample=3115, ActualDelta=2.5 cm	事实反馈
9	Mesh RefreshBoneTransforms	Evaluate Start 的最终骨骼 Pose, 写 PoseHistory	Pose Evaluate

第 6 步足以给第 7 步提供 Root Motion; 第 9 步不需要倒流回 CMC。它们共享同一轮播放器状态, 但属于不同计算产品。

13 实例 B: 持续按 W 撞墙

撞墙不改变上述调度, 只改变 CMC 结果。MM Update 仍可选择 Run Loop 并提取向前 Root Motion; CMC 随后把请求位移裁剪为接近 0; Evaluate 仍根据当前 BlendStack 生成 Run Pose。下一帧 Query 同时看到“Intent 仍为 Run”和“上一轮实际位移很小”。

信号	值	在哪个阶段形成
TargetIntentVelocity	Run 标称速度, 向前	BuildMotionMatchingQuerySnapshot
ExtractedRootMotion	动画向前 Delta	AnimGraph Update / Asset Player Tick
FinalDeltaWS	Coordinator 后的唯一请求位移	CMC Root Motion 转换阶段
ActualTranslationDelta	接近 0	CMC 碰撞完成后
Visible Pose	通常仍为 Run	移动后的 Pose Evaluate
下一帧 Query	历史近 0, 未来仍向前	下一次 NativeUpdateAnimation

14 哪些情况下时序会变

情况	Animation Update	Evaluate	说明
本项目 RootMotionFromEverything	CMC 移动前强制立即完成	Mesh Tick 后续发起, 可并行	本文主路径
非 Root Motion Character	CMC 通常在移动后 TickCharacterPose; 或由 Mesh 调度	Mesh RefreshBoneTransforms	不需要移动前 Root Motion 同步
普通 SkeletalMeshActor	自身 Mesh Component Tick	同一组件 Tick 内随后发起, 但仍是独立阶段	没有 CharacterMovement 控制
网络模拟 / Replay	SimulatedTick、校正与重放可能多次 Autonomous Tick	仍受 Mesh / 并行任务控制	不能只用单一本地帧图解释
URO / 不可见	Update 或 Evaluate 可跳帧、插值或只 Tick Montage	可能复用缓存	ShouldTickAnimation / VisibilityBasedAnimTickOption
蓝图断点 / 关闭并行动画	可能强制游戏线程立即 Update	可能回到游戏线程 Evaluate	调试时线程与 Shipping 不同

所以文档只能说“本项目当前配置下的主路径”, 不能说 Animation Update 与 Evaluate 总是一起, 也不能说所有动画工作都发生在 Movement Tick 内。

15 修正版逐帧时序表

序	调度边界	调用 / 工作	线程与时点	是否移动 Actor
1	项目组件	InputManager Tick	GT / PrePhysics	否
2	项目 Movement	Lua UpdateMovementLogic → Submit Intent	GT / CMC Super 前	否
3	CMC	PerformMovement → TickCharacterPose	GT / 物理前	否
4	AnimInstance	PreUpdate + NativeUpdateAnimation + BlueprintUpdate	GT / CMC 同步请求内	否
5	AnimGraph Update	MM Search + BlendStack Update	本项目被强制 GT 立即完成; 一般可并行	否
6	Asset Player Tick	推进时间并提取 Root Motion	Update 尾部	否
7	AnimInstance PostUpdate	Proxy RM → UAnimInstance 缓冲	GT / 立即 Update 尾部	否
8	CMC	Mesh.ConsumeRootMotion → Convert → Coordinator	GT / PerformMovement	否
9	CMC Physics	RM Velocity → PhysWalking / Collision	GT / PerformMovement	是
10	完成回调	CaptureCompletedMovement	GT / 移动完成	只记录
11	Mesh Component Tick	RefreshBoneTransforms 发起 Evaluate	Movement prerequisite 之后	否
12	Pose Evaluate	Evaluate_AnyThread → Bones / IK / PoseHistory	常见 Worker; 也可 GT	否
13	完成任务	PostAnimEvaluation / buffer swap / finalize	GT completion	否

16 断点验证：怎样亲自确认这三个阶段

建议分别设置三组断点，而不是只断在 `TickCharacterPose`：

```

A. Update 与 Root Motion 同步点
UCharacterMovementComponent::TickCharacterPose
UAnimInstance::UpdateAnimation
USKMotionMatchingAnimInstance::NativeUpdateAnimation
FAnimNode_MotionMatching::UpdateAssetPlayer
FAnimInstanceProxy::TickAssetPlayerInstances
UAnimInstance::PostUpdateAnimation
USkeletalMeshComponent::ConsumeRootMotion_Internal

B. CMC 物理
UCharacterMovementComponent::ConvertLocalRootMotionToWorld
USKMovementComponent::CoordinateRootMotionPostConvert
UCharacterMovementComponent::PhysWalking
USKMotionMatchingTrajectoryComponent::CaptureCompletedMovement

C. Pose Evaluate
USkinnedMeshComponent::TickComponent
USkeletalMeshComponent::RefreshBoneTransforms
USkeletalMeshComponent::ParallelAnimationEvaluation
FAnimNode_MotionMatching::Evaluate_AnyThread
FAnimNode_PoseSearchHistoryCollector::Evaluate_AnyThread
  
```

在 Rider 的 Threads / Call Stack 中应看到：A 组嵌套在 CMC 调用栈；B 组随后改变胶囊；C 组从 Mesh Tick / 动画任务进入，并可能出现在工作线程。开启动画蓝图断点可能迫使 Update / Evaluate 回到游戏线程，因此性能与时序分析优先使用 Unreal Insights。

17 项目代码流程索引

职责	项目 / 引擎文件	入口
输入意图	Content/Script/Gameplay/Sekiro/Input/SKInputManager.lua	OnMoveInput
正式 Intent	Content/Script/Gameplay/Sekiro/Movement/SKMovementComponent.lua	UpdateMovementLogic
项目 Movement 外层	Source/Sekiro/Movement/SKMovementComponent.cpp	TickComponent / RootMotion callbacks
Query 与实际反馈	Source/Sekiro/Movement/SKMotionMatchingTrajectoryComponent.cpp	BuildQuery / CaptureCompletedMovement
Phase / Chooser / 邮箱	Source/Sekiro/Animation/SKMotionMatchingAnimInstance.cpp	NativeUpdate / SearchRequest / PostSelection
AnimGraph 结构	Content/Script/Animation/SekiroMotionMatching/ABP_SekiroMotionMatching.lua	AnimGraph
Character Tick 依赖	Engine/Source/Runtime/Engine/Private/Character.cpp	PostInitializeComponents
CMC 同步消费	Engine/.../Private/Components/CharacterMovementComponent.cpp	PerformMovement / TickCharacterPose
Update / Evaluate 分离	Engine/.../Private/Components/SkeletalMeshComponent.cpp	TickPose / RefreshBoneTransforms
AnimInstance 立即更新	Engine/.../Private/Animation/AnimInstance.cpp	UpdateAnimation / NeedsImmediateUpdate
MM 两阶段	Engine/Plugins/Animation/PoseSearch/.../AnimNode_MotionMatching.cpp	UpdateAssetPlayer / Evaluate_AnyThread

18 结论

本项目的准确时序不是“CMC 调用动画 Update + Evaluate，然后消费 Root Motion”。准确描述是：CMC 因 RootMotionFromEverything 在物理前请求 Animation Update；游戏线程前置更新构建 Query，立即 AnimGraph Update 完成 Motion Matching 搜索、播放器推进和 Root Motion 提取；CMC 消费这份结果并完成碰撞移动；之后 Mesh Component Tick 才通过 RefreshBoneTransforms 发起 Pose Evaluate，常见情况下在工作线程计算最终骨骼与 PoseHistory。

一句话记忆：Update 决定播放状态并产生可消费的时间区间 Root Motion；CMC 用它移动胶囊；Evaluate 随后计算最终骨骼 Pose。三者有关联，但不是同一步。

范围声明：上述主时序针对当前 Sekiro5.8 的 RootMotionFromEverything 本地 Character 路径；网络模拟、URO、不可见 Mesh、Montage-only 与调试强制单线程分支已在第 14 节单列。